Managing Microservices with Kubernetes and Istio

2024 edition

Pearson

Intro

- This session is provided by Sander van Vugt
- It is a completely reworked class, based on my on-demand class "Managing Microservices with Containers, Kubernetes and Istio"
- Today, Friday Feb. 23rd is the first time I'm running this completely reworked class, it may contain bugs
- Participants are expected to know how to run an application in Kubernetes
- To follow along, use Minikube inside an Ubuntu Desktop environment
- Other environments may work, but are not supported
- Due to the high density of information in this class, you may also consider to just watch and use the recordings later to follow along

Poll Question 1

Rate your Kubernetes knowledge

- none
- poor
- average
- good
- more than good

Poll Question 2

- Where are you from?
- Middle East
- Africa
- India
- Asia (other)
- North/Central America
- South America
- Pacific region
- Europe

Agenda

- Understanding Microservices
- Container-based Microservices
- Microservices in Kubernetes
- Why using GitOps makes sense
- What Istio may add to your solution
- Running your Microservice application in Istio

Using an environment in this course

- Recommended: create an Ubuntu-based minikube environment as described in the setup guide in https://github.com/sandervanvugt/microservices
- Alternatively: use O'Reilly sandbox, but functionality may be missing or different

Understanding Microservices

What are Microservices?

- A microservice is an application that consists of different independent components that are connected together
- The independent components can be individually developed by completely disconnected projects
- An additional layer is used to add site-specific information to the microservices

Why use Microservices?

- Developing independent small applications is easier than maintaining one big application
- Changes can be applied faster
- By applying common DevOps technologies like Continuous Integration / Continuous Development, zero-downtime application upgrades can be provided

Microservices Benefits

- When broken down in pieces, applications are easier to build and maintain
- Each piece is independently developed, and the application itself is just the sum of all pieces
- Smaller pieces of application code are easier to understand
- Developers can work on applications independently
- If one component fails, you spin up another while the rest of the application continues to function
- Smaller components are easier to scale
- Individual components are easier to fit into continuous delivery pipelines and complex deployment scenarios

The minimal approach: Compose

Understanding Dockerfile

- Dockerfile is a way to automate container image builds
- In Red Hat environments, "Containerfile" is used instead of "Dockerfile"
- It contains all instructions required to build a container image
- Use **docker build . -t docker.io/yourname/myapp:1.0** to build the container image based on the Dockerfile in the current directory and tag it for storage on docker.io
- Images will be stored on your local system, and provide the right tag to facilitate uploading it to Docker hub
- Tip: many images on hub.docker.com have a link to Dockerfile. Read it to understand how an image is built using Dockerfile!

Demo: Using a Dockerfile

- Dockerfile demo is in the course Git repository at https://github.com/sandervanvugt/gitops/dockerfiledemo
- Use **docker build -t docker.io/yourname/nmap:1.0 .** to run it from the current directory
- Tip: use **docker build --no-cache -t nmap .** to ensure the complete procedure is performed again if you need to run it again
- Next, use **docker run nmap** to run it
- Use **docker push docker.io/yourname/nmap:1.0** to push it to Docker hub

Containers and Microservices

- A container is an application that runs based on a container image. The container image is a lightweight, standalone, executable package of software that includes everything that is needed to run an application
- Because containers are lightweight and include all dependencies required to run an application, containers have become the standard for developing applications
- As containers are focusing on their specific functionality, they are perfect building blocks for building microservices
- In a containerized microservices environment, site-specific information can be provided in a flexible way by using orchestration tools such as Kubernetes

From Code to Containerized Microservice

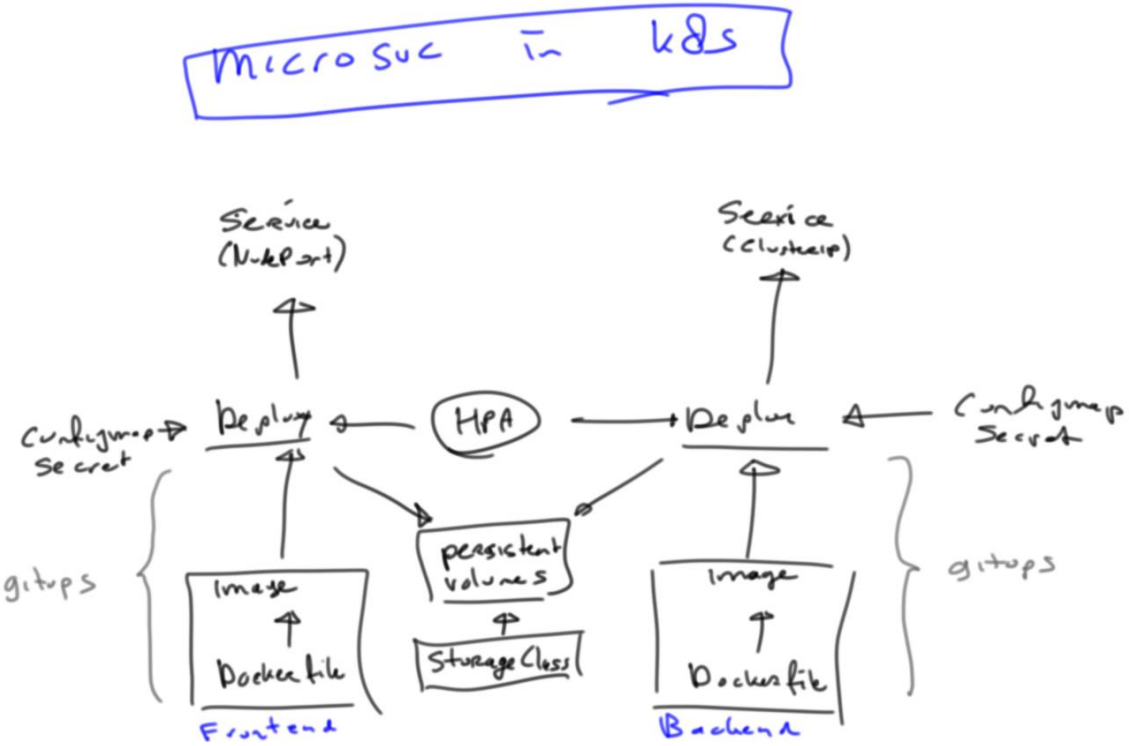
- Optionally, create Dockerfile to build custom container images
- Manage the development and versions of the Dockerfile by using GitOps
- Upload the resulting customized images to a container registry
- From there, use Compose to run the microservice with specific properties
  - **docker-compose**
  - **podman-compose**

Demo: Running a docker-compose Microservice

- **sudo apt update**
- **sudo apt install docker-compose**
- **cat microservices/docker-compose**
- **docker-compose up**

Microservices in Kubernetes

NTS


Microsvc in k8s
Service (NodePort)
Service (ClusterIP)
configmap secret
Deploy
HPA
Deploy
Configmap Secret
gitops
Image
Dockerfile
Frontend
persistent volumes
StorageClass
Image
Dockerfile
Backend
gitops

Commonly Used Resources

- Deployment: most common resource for running applications
- StatefulSet: particularly recommended for running statefule applications such as databases
- Pods: application instances, should be scheduled by Deployments
- Services: work like a load-balancer to provide access to applications
- ConfigMaps: used to store configuration in the cluster and dynamically connect to the applications
- Secret: base-64 encoded alternative for the ConfigMap
- PersistentVolume: represents storage in the cluster. Often automatically provisioned
- PersistentVolumeClaim: used by applications to connect to PersistenVolumes

Demo: Running a Kubernetes Application

- This demo is improvised. You can find the exact commands in this demo in the course command history, which will be published on https://pastebin.com/u/sandervanvugt after the course.

Understanding **Kustomize**

- Kustomize is a Kubernetes feature that uses a file with the name kustomization.yaml to store instructions for the changes a user wants to make to a set of resources
- This is convenient for applying changes to input files that the user does not control himself, and which contents may change because of new versions appearing in Git
- So it's a way to modify existing configuration files, and thus implement changes in a declarative way
- Use **kubectl apply -k ./** in the directory with the **kustomization.yaml** and the files it refers to to apply changes
- Use **kubectl delete -k ./** in the same directory to delete all that was created by the Kustomization

Demo: Using Kustomization

- **cat deployment.yaml**
- **cat service.yaml**
- **kubectl apply -f deployment.yaml service.yaml**
- **cat kustomization.yaml**
- **kubectl apply -k .**

Demo: Microservice on Kubernetes

- In this demo, we'll create a microservice that consists of a Wordpress application and a MySQL application, that meets the following requirements
- Both applications are using a PVC to refer to the storage that is offered by using the default StorageClass
- Create one yaml file for each, such that it is easy to refer to the appropriate resources from the kustomization.yaml
- A kustomization.yaml is used to generate a secret that contains the MySQL root password as well as a configMap that specifies the name of the Wordpress host as a variable
- The kustomization.yaml is used to refer to the WordPress as well as the MySQL resource configurations

Demo: Verifying the Results

- **lesson6lab/kustomization.yaml**
- **lesson6lab/wordpress-deployment.yaml**
- **lesson6lab/mysql-deplyment.yaml**
- **kubectl get secrets**
- **kubectl get deployments**
- **kubectl get pvc** (will take few minutes)
- **kubectl get pods** (wait until all are running)
- **kubectl get services wordpress**
- **minikube service wordpress --url**
- From Minikube host: http://$(minikube ip):$(serviceport)

Why Using GitOps Makes Sense in Microservices

What is GitOps

- GitOps is using DevOps technology like CI/CD pipelines to manage versioning and application updates of IaC manifests that are stored in a Git repository
- To do so efficiently, the GitOps controller monitors the Git repository, picks up changes to the code and (semi-)automatically updates running applications
- GitOps can be used on different components of the Microservice to ensure that individual components get updated when needed
- Check "Automating Kubernetes with GitOps" for more details (https://learning.oreilly.com/course/automating-kubernetes-with/9780138230258/)

What Istio adds to the Solution

What is Istio

- Istio is a service mesh
- Other service mesh solutions exist, like LinkerD
- A service mesh adds traffic controller options to Kubernetes
- To do so, it uses Kubernetes API extensions to add resources that integrate in Kubernetes
- To mananage cluster traffic, Istio injects an envoy proxy as a sidecar container in each Istio managed application Pod

Should I use Istio?

- Adding Istio to manage microservices adds functionality but also complexity
- Common benefits that Istio adds
  - Securing Service to Service traffic with mutual TLS
  - Complex traffic management
  - Deployment scenario's like blue/green and canary
  - Observability and monitoring using integrated Grafana and Prometheus
  - Policy enforcement like rate limitations, access control and quota

Reasons not to Use Istio

- Istio adds significant overhead
- Many features like Blue/green deployment and canary deployments are also available without Istio
- Grafana and Prometheus can be added to Kubernetes without using istio

Istio Quickstart

Getting Started with Istio

- The Istio documentation contains a getting started guide for the current version of Istio: https://istio.io/latest/docs/setup/getting-started/
  - Consult it for details about requirements in specific Kubernetes cluster environments
- To get started with Istio, the following steps need to be performed
  - Download Istio
  - Install Istio
  - Deploy the sample application
  - Open the application to outside traffic
  - View application behaviour in the Dashboard

Understanding Istio Profiles

- Istio profiles determine the type of functionality that is installed
  - The **demo** profile is suitable for exploring Istio with minimal system requirements and high levels of tracing and access logging
  - The **default** profile is recommended for production environments
  - The **minimal** profile is setting up the control plane only
  - Specific platforms (such as OpenShift) have their own profile

Lab: Download and Install Istio

- Clean up all current configuration: **kubectl delete all -A**
- **curl -L https://istio.io/downloadIstio | sh -**
- **cd istio-[Tab]**
- **sudo cp bin/istioctl /usr/bin/**
- **istioctl install --set profile=demo -y**
- **kubectl get all -n istio-system**
- **kubectl label namespace default istio-injection=enabled**
- **kubectl get crds**

Istio Key Resources: Gateways

- istio-ingress-gateway: a Kubernetes Service, deployed in the istio-system namespace. Part of the Istio control plane, it is the entry point for external traffic into the cluster.
- As it is a Kubernetes Service, external loadbalancers forward traffic with a destination to the istio-ingress-gateway, which further handles the traffic using Istio Gateway and VirtualService resource
- Gateway: contains the configuration that is used by the istio-ingress-gateway to forward traffic to a specific application (represented by its service)
- Each Gateway resource has a Hosts field, which tells the istio-ingress-gateway to witch Kubernetes Service the traffic should be forwarded

Istio Key Resources: VirtualServices

- VirtualService resource implement Istio traffic routing
- VirtualService resources allow to distribute traffic to different versions of an application, using the **subset** property for a **destination** (in which case a DestinationRule is needed as well)
- This makes them perfect for implementing Canary Deployments
- The Subset property can be configured to send a specific percentage of the inbound traffic to a specific version of the application
- VirtualServices can also be used to direct traffic to multiple different destination Kubernetes Services
- VirtualServices can use filtering conditions that are based on header information in the inbound traffic, such as usernames

Istio Key Resources: DestinationRules

- The DestinationRule applies a policy after traffic has been routed to a specific application
- The DestinationRule defines subsets of Kubernetes Services based on labels set on these Services
- It also specifies the load balancing policy, such as round-robin, random, weighted and others
- You can also use it to specify TLS settings and other properties

NTS: overview

Understanding Istio Applications

- Kiali: the observability console for Istio
- Grafana: an open source multi-platform analytics and visualization web application
- Prometheus: records real-time metrics used for event monitoring and alerting
- Jaeger: software for tracing transactions between distributed versions

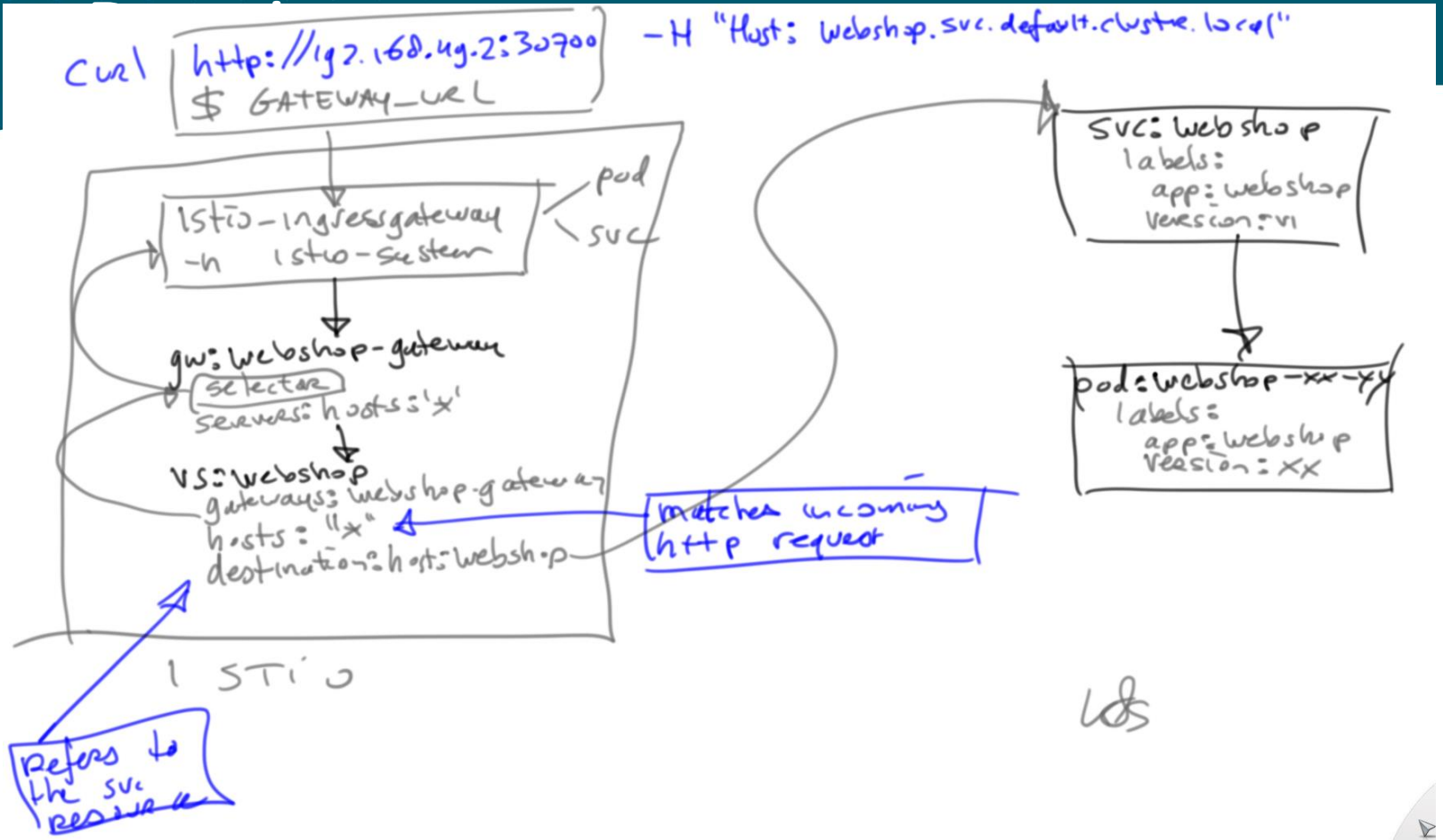
Running a Simple Microservice in Istio

Understanding Istio Requirements

- You'll need an istio-ingressgateway and related resources running in the istio-system namespace
- For use in Minikube, configure the istio-ingressgateway service as a NodePort type Service
- Create an Istio Gateway resource in the namespace that runs the application to catch all incoming traffic for that specific application
- Create an Istio VirtualService with three properties:
  - gateways: refers to the Istio Gateway running in the application namespace
  - hosts: matches the incoming HTTP request. Set to "*" to make it easier
  - destination: refers to the Kubernetes Service resource

Understanding Kubernetes Requirements

- A ConfigMap or PersistentVolume
  - Needed to provide version specific information
- The application Service resource
  - Needs a selector label to select all Pod versions
  - Needs a version label to specify the version needed for Istio Canary Deployment
- The application Deployment resource
  - One Deployment for each version
  - A specific version label for each version
  - Mounts a ConfigMap with specific version information


curl
http://192.168.49.2:30700
$ GATEWAY_URL
-H "Host: webshop.svc.default.cluster.local"
istio-ingressgateway
-n istio-system
pod
svc
gw: webshop-gateway
selector
servers: hosts: '*'
VS: webshop
gateways: webshop-gateway
hosts: "*"
destination: host: webshop
matches incoming
http request
svc: webshop
labels:
app: webshop
version: v1
pod: webshop-xx-yy
labels:
app: webshop
version: xx
ISTIO
refers to
the svc
resource
LB

Understanding the Demo

- First, Istio resources are added to make the Kubernetes application accessible through the istio-ingress-gateway resource
- Next, a new Virtual Service resource is added to manage Canary Deployment traffic in specific proportions. This Virtual Service uses the Subsets that are defined in the DestinationRules, which identify specific Service versions by the label property

Demo: Running your Own Application

- **cd ~/microservices/myapp-test/**
- **kubectl apply -f version-{one|two|three}-cm.yaml**
- **kubectl apply -f version{1|2|3}-deploy.yaml**
- **kubectl apply -f webshop-svc.yaml** #version label is unneeded
- **kubectl get svc**
- **curl $(minikube ip):32000** # should give different versions
- **kubectl get all -n istio-system** # should be up and running
- **echo $GATEWAY_URL || source setup_istio_vars.sh**
- **kubectl apply -f webshop-gateway.yaml**
- **kubectl apply -f virtual-service-initial.yaml**
- **curl $GATEWAY_URL -H "Host: webshop.svc.default.cluster.local"** # multiple times to verify RR LB

Demo: Running your Own Application

- **istioctl proxy-config routes deploy/istio-ingressgateway.istio-system**
- **istioctl proxy-config listener deploy/istio-ingressgateway.istio-system**
- **kubectl apply -f virtual-service-webshop-80-20.yaml**
- **kubectl apply -f destination-rule-webshop.yaml**
- **curl $GATEWAY_URL -H "Host: webshop.svc.default.cluster.local"** # (repeat multiple times)

Demo: Using the Dashboard

- **kubectl apply -f samples/addons**
- **kubectl rollout status deployment/kiali -n istio-system**
- **istioctl dashboard kiali**
- In the left navigation menu, select Graph, and in the Select Namespace drop-down, select default (may have to resize screen)
  - You won't see anything before traffic hits the mesh
- Set Replay to 5 Min and Refresh Interval to every 10 seconds
- Use **for i in $(seq 1 100); do curl -s -o /dev/null http://$GATEWAY_URL -H "Host: webshop.svc.default.cluster.local"; done** to generate some traffic and observe what is happening in the Dashboard

Optional Demo: Exploring Prometheus

- **kubectl apply -f samples/addons/prometheus.yaml**
- **kubectl -n istio-system get svc prometheus**
- Generate some traffic to the application: **curl http://$GATEWAY_URL -H "Host: webshop.svc.default.cluster.local"**
- **istioctl dashboard prometheus**
- From the Dashboard, click **Graph**, and in the Expression input box, enter the text **istio_requests_total**. Next, click **Execute** to see an overview of all requests

Further learning

- More information is in the on-demand course "Managing Microservices with Containers, Kubernetes and Istio", available on this platform
- Also check the RESOURCEX.TXT file in this course Git repository at https://github.com/sandervanvugt/microservices